

THAW A CREATURE

THAW A CREATURE

Technical Implementation Specification - 3-Week Public Launch - Canonical v0.5

Platform: Roblox

Target: 3-week public launch

Multiplayer Target: 4 players/server initially

Architecture: Server-authoritative, configuration-driven

Primary Design Source: Concise GDD v0.5

Revision notes

- Normalized all config schemas and naming to match Constants.
- Locked exact Great Frost wall-clock/boot/transition behavior and Grab locking.
- Locked Special Ice lifecycle on failure, disconnect, and cycle expiry.
- Locked shared-camp base topology, visitor Warmth behavior, and private onboarding Ice.
- Locked no intentional raw-Ice drop action and exact Freeze/respawn recovery.

v0.5 Architectural Revision - Per-Copy Size Jackpot

This revision deliberately supersedes the earlier species-count ownership and rarity-only fixed Speed/Income model. Public-launch architecture must now support unique CreatureGUID copies that inherit one Ice size/weight roll. Scale is derived from that roll and modifies copy-specific Pen Income and Equipped Speed. The revision occurs before persistence implementation so the saved-data schema can be correct from the start.

1. Purpose

This document defines how the canonical game design should be implemented in Roblox. The implementation must support the core fantasy:

See it. Risk it. Bring it home. Watch it thaw. Smash it open. The technical architecture must prioritize:

1. reliable expedition gameplay;
2. secure server authority;
3. simple configuration;
4. persistence;

5. multiplayer safety;
6. rapid content addition;
7. strong presentation hooks for the five Hero Moments. The implementation should not introduce additional gameplay systems, currencies, progression tracks, or abstractions unless specified by the design documents.

2. Core Technical Principles

Server Owns Gameplay Truth The server is authoritative for: Cash;

permanent Maximum Warmth; Current Warmth; Hearth Level; Warmth training; Equipped creature; Movement Speed; creature ownership; Pen contents; passive income; Ice existence; Ice ownership; Ice creature result; rarity result; thaw progress; Great Frost cycle gameplay; Special Ice; reveal state; Frozen Archive; selling; persistence. The client handles: UI; input; camera; sounds; particles; animation; reveal presentation; low-Warmth presentation; Great Frost presentation; creature follower movement; Pen creature roaming; movement-speed presentation.

The client may request actions. The client never decides their results.

3. Primary Architecture

```

1  ReplicatedStorage
2  |
3  +-- Shared
4  |   +-- GameConfig
5  |   +-- CreatureConfig
6  |   +-- RarityConfig
7  |   +-- IceConfig
8  |   +-- WarmthConfig
9  |   +-- HearthConfig
10 |   +-- WorldCycleConfig
11 |   +-- RegionConfig
12 |
13 +-- Remotes
14 |   +-- GameplayRequest
15 |   +-- GameplayEvent
16 |
17 +-- Assets
18     +-- IceVisuals
19     +-- Silhouettes
20     +-- Effects
21     +-- WorldEffects
22     +-- UI
23
24
25 ServerStorage
26 |
27 +-- CreatureModels
28     +-- Penguin
29     +-- Rabbit
30     +-- ...
31
32
```

```

33  ServerScriptService
34  |
35  +-- ServerBootstrap
36  |
37  +-- Services
38      +-- DataService
39      +-- PlayerStateService
40      +-- BaseService
41      +-- WarmthService
42      +-- WorldCycleService
43      +-- IceService
44      +-- ThawService
45      +-- CreatureService
46
47
48  StarterPlayer
49  +-- StarterPlayerScripts
50      +-- ClientBootstrap
51      |
52      +-- Controllers
53          +-- InteractionController
54          +-- WarmthController
55          +-- WorldCycleController
56          +-- ThawController
57          +-- CreatureVisualController

```

```

58          +-- MovementPresentationController
59          +-- EffectsController
60          +-- UIController
61
62
63  Workspace
64  |
65  +-- Wilderness
66  |
67  +-- IceSpawnPoints
68      | +-- Regular
69      | +-- Thick
70      | +-- Ancient
71      | +-- Black
72      | +-- Meteor
73      |
74  +-- PlayerBases
75      | +-- BaseSlot01
76      | +-- BaseSlot02
77      | +-- BaseSlot03
78      | +-- BaseSlot04
79      |
80  +-- WorldFXAnchors

```

4. Configuration-Driven Content

Gameplay numbers must not be scattered through scripts. All balancing values belong in configuration modules.

CreatureConfig

Each species defines at minimum:

```
CreatureID  
DisplayName  
Rarity  
ModelName  
SilhouetteProfile  
ReferenceWeight  
BaseVisualScale  
BaseIncomePerSecond  
BaseWalkSpeed
```

Species now owns baseline economy/traversal identity. Rarity may still influence presentation, pools, sell baselines, and rarity treatment, but it no longer uniquely determines Pen Income or Equipped Speed.

RarityConfig

Defines supported rarity presentation/economy tiers. The current economy supports:

```
Common  
Uncommon  
Rare  
Epic  
Legendary
```

Epic/Legendary content assignment is separate from the validity of those configured economy tiers. Do not infer creatures/pools/weights merely because the tier exists.

RarityConfig may define:

- SellValue baseline unless species-specific sell is later approved;
- RevealPresentationProfile;
- RarityUI;
- EffectsProfile;
- fallback migration values only when explicitly needed.

SizeConfig

Defines the one canonical size-roll system:

```
MinVisualScale  
MaxVisualScale  
SizeBands  
WeightMultiplier ranges  
WithinBandBias  
IncomeScaleExponent  
SpeedScaleSlope
```

Runtime formula:

```
ActualWeight = ReferenceWeight * WeightMultiplier  
Scale = cbrt(ActualWeight / ReferenceWeight)
```

Because the ratio reduces to `WeightMultiplier`, `Scale` can also be derived as `cbrt(WeightMultiplier)` while retaining the explicit weight model.

IceConfig

Each Ice Type defines:

```
IceType  
RarityWeights  
CreaturePoolsByRarity  
BaseThawDuration  
CarryDrainMultiplier  
PresentationProfile  
ReferenceWeight  
BaseVisualScale  
RegionID
```

Launch types remain:

```
Regular  
Thick  
Ancient  
Black  
Meteor
```

Every spawned Ice gets a server-authoritative `WeightMultiplier`. `Scale` is derived from the formula and is independent of hidden rarity/species unless a later explicit design change says otherwise.

WarmthConfig / RegionConfig

`WarmthConfig` defines starting progression and base drain/refill behavior. `RegionConfig` defines:

```
RegionID  
DisplayName  
RecommendedWarmth  
ColdMultiplier
```

Recommended Warmth is informational only and must never become an access gate.

HearthConfig

Each Hearth level defines:

```
Level  
UpgradeCost
```

The Hearth never defines a Maximum Warmth cap.

5. Persistent Player Data

Public launch requires persistent data. Recommended schema:

```
DataVersion = 2

Cash
MaxWarmth
HearthLevel

CreatureCopies = {
  [CreatureGUID] = {
    CreatureID,
    WeightMultiplier,
  }
}

PenSlots = {
  [1] = CreatureGUID or nil,
  [2] = CreatureGUID or nil,
  [3] = CreatureGUID or nil,
  [4] = CreatureGUID or nil,
}

EquippedCreatureGUID = CreatureGUID or nil

DiscoveredCreatures = {
  [CreatureID] = true
}

ThawingIce = {
  {
    ThawID,
    IceType,
    IsSpecial,
    CreatureID,      -- server-only hidden reward
    Rarity,          -- server-only hidden reward
    WeightMultiplier, -- inherited from wilderness Ice
    DepositedAtUnix,
    BaseDurationSeconds,
    BonusProgressSeconds,
  }
}

PityState = {
  CommonStreak = number
}
```

Scale, FinalIncome, and FinalWalkSpeed should normally be **derived**, not redundantly persisted:

```
Scale = cbrt(WeightMultiplier)
FinalIncome = f(CreatureConfig[CreatureID].BaseIncomePerSecond, Scale)
FinalWalkSpeed = g(CreatureConfig[CreatureID].BaseWalkSpeed, Scale)
```

This keeps each individual creature record compact while preserving unique jackpot identity.

Data migration from the old species-count prototype representation must be explicit. Do not silently mix count-based ownership and GUID-based ownership after the migration task is complete.

6. Data That Must NOT Persist

Do not persist: Current Warmth; carried Ice; current character position; active reveal session; temporary Frozen state; current Great Frost phase; world Ice ownership; follower position; Pen creature visual positions. On join:

CurrentWarmth = MaxWarmth The player starts safely at their base.

7. Unlimited Thawing - Data Rule

There is no player-facing Thaw capacity. ThawingIce therefore contains an arbitrary number of logical Ice records. Do not create: MaxThawSlots ; purchasable thaw capacity; separate incubator slots. The persistent records must remain compact. Do not store: Models; CFrames;

visual state; particles; redundant creature metadata. Only gameplay data should persist. An internal technical safety guard may protect against impossible/exploit-generated data volumes, but this must never function as normal gameplay progression.

8. Player Runtime State

PlayerStateService maintains non-persistent session state such as:

```
1  CurrentWarmth
2  IsInsideOwnWarmZone
3  IsTrainingWarmth
4
5  CurrentlyCarryingIceID
6
7  ActiveRevealSession
8
9  AssignedBase
10 IsFrozen
11
12 LastGameplayRequestTimes
13
14 CurrentMovementSpeed
```

Persistent values are accessed through the player's loaded profile.

9. Warmth System

Warmth has two values:

```
1 CurrentWarmth
2 MaxWarmth
```

MaxWarmth is permanent. CurrentWarmth is session state.

Wilderness While outside the player's own Warm Zone:

```
1 CurrentWarmth -= DrainRate * dt
```

While carrying Ice:

```
1 CurrentWarmth -= DrainRate
2                 * CarryMultiplier
3                 * dt
```

No creature directly increases Warmth.

Warm Base While inside the player's own Warm Zone:

```
1 CurrentWarmth += RefillRate * dt
```

up to:

```
1 MaxWarmth
```

At the same time:

```
1 MaxWarmth += HearthTrainingRate * dt
```

Warmth training does not stop at a cap.

10. Warmth Training

Warmth training is automatic. The player does not send:

```
1 TrainWarmth
```

requests. The server determines:

```
1 player is inside own WarmZone
```

and awards training continuously. The training rate is:

```
1 HearthConfig[HearthLevel].TrainingRate
```

There is: no XP bar; no Warmth currency; no diminishing-return formula; no maximum trainable value. The primary UI should simply show the Warmth number increasing. Fractional progress may be stored internally while the UI displays a rounded/floored value.

11. Hearth Upgrades

UpgradeHearth is a server-authoritative purchase. Server validates:

```
1 player has loaded data
2 player is at own base
3 next level exists
4 player has enough Cash
```

Then:

```
1 Cash -= UpgradeCost
2 HearthLevel += 1
```

The new training rate applies immediately. Hearth upgrade does not directly add Warmth unless later explicitly configured. Its primary function is:

faster permanent Warmth training.

12. Recommended Warmth

Recommended Warmth is not gameplay validation. It exists for: region signs; UI; progression communication; onboarding. Example:

```
1 ANCIENT WILDERNESS
2 Recommended Warmth: 220
```

The server does not prevent a 250-Warmth player from entering.

13. Equipped Creature Speed

Equipping a creature changes actual character Movement Speed. Speed is determined from the specific equipped CreatureGUID:

```
copy = CreatureCopies[EquippedCreatureGUID]
base = CreatureConfig[copy.CreatureID].BaseWalkSpeed
scale = cbrt(copy.WeightMultiplier)
FinalWalkSpeed = base * SizeSpeedMultiplier(scale)
```

Size affects traversal more gently than Pen Income. A maximum-scale creature should be dramatically better without turning movement into teleportation or removing Warmth strategy.

The client may amplify the sensation through presentation, but the actual WalkSpeed is server-authoritative.

14. Speed Authority

On: character spawn; Equip; Unequip; creature replacement; the server recalculates:

```
1 CurrentMovementSpeed
```

and applies it to the character. The client never submits its desired Speed. Gameplay interactions must use server-observed character position, not coordinates claimed by the client.

15. Creature Equip Rules

One individual CreatureGUID may be Equipped.

Equip from Inventory:

```
Inventory CreatureGUID -> EquippedCreatureGUID
```

Equip from Pen:

```
clear Pen slot
CreatureGUID -> EquippedCreatureGUID
```

Replacing Equipped creature:

```
old EquippedCreatureGUID -> Inventory
new CreatureGUID -> EquippedCreatureGUID
```

Unequip:

```
EquippedCreatureGUID -> Inventory
```

No automatic Pen refill occurs. Equip/Unequip should occur while at the player's own base. All validation uses CreatureGUID ownership, not merely CreatureID/species count.

16. Ice Spawn Architecture

Do not procedurally spawn Ice inside arbitrary world volumes. Use handcrafted spawn points.

```
1 Workspace
2 +-- IceSpawnPoints
3   +-- Regular
4     | +-- Spawn01
5     | +-- Spawn02
6     | +-- ...
7   +-- Thick
8   +-- Ancient
9   +-- Black
10  +-- Meteor
```

Each SpawnPoint defines where Ice is allowed to appear. This gives level design control over: visibility; route choice; silhouette framing; Warmth distance; Special Ice presentation. The game randomly chooses among designed locations, rather than randomly generating arbitrary coordinates.

17. Ice Runtime Record

Each wilderness Ice record contains at minimum:

```
IceID
IceType
SpawnPointID
State
OwnerUserID or nil

-- Hidden reward data
CreatureID
Rarity

-- Visible jackpot lineage
WeightMultiplier
```

Derived:

```
Scale = cbtr(WeightMultiplier)
```

`WeightMultiplier` is rolled once when the Ice opportunity is created. It must never be rerolled during carry, deposit, thaw, or reveal.

The client may know visible Scale because the Ice visibly advertises it, but the client must not receive hidden reward identity before reveal.

World Ice visuals should be non-colliding or otherwise isolated from traversal physics so extreme 5x presentation does not become an accidental hard wall. Interaction validation remains server-side and should not depend on oversized visual geometry alone.

17A. Size-Roll Authority

The server samples a configured weight-multiplier band and derives Scale using:

```
ActualWeight = ReferenceWeight * WeightMultiplier  
Scale = cbrt(ActualWeight / ReferenceWeight)
```

The current maximum supported visual Scale is 5.0x. Rare extreme rolls are intentional jackpot events.

The size roll is independent of rarity/species by default. A giant Ice therefore signals exceptional size/value without exposing the exact hidden creature.

18. When the Creature Result Is Rolled

Creature outcome is selected by the server when the wilderness Ice is spawned. Pipeline:

```
1  Ice Type  
2      v  
3  Special/Normal Roll Profile  
4      v  
5  Bad-Luck Protection if enabled  
6      v  
7  Rarity Roll  
8      v  
9  Eligible Creature Pool  
10     v  
11  Creature Roll  
12     v  
13  Store hidden result  
14     v  
15  Select matching/shared SilhouetteProfile
```

This allows the physical Ice to contain an appropriate silhouette before collection.

19. Silhouette Privacy

The client may receive:

```
1 SilhouetteProfile
```

but not:

```
1 CreatureID
2 Rarity
```

where possible.

Multiple creatures should be able to share or reuse silhouette profiles. The goal is:

hint without reliable exact identification.

Use the canonical SilhouetteProfile IDs/mapping from Implementation Constants. The client should receive only the profile, never exact CreatureID/Rarity before approved reveal information.

20. Grabbing Ice

Player interacts with an Ice using a ProximityPrompt or equivalent world interaction. Client requests:

```
1 GrabIce(IceInstanceID)
```

Server validates: Ice exists; state is Available ; player is alive; player is not Frozen; player carries no other Ice; server-measured proximity is valid; Ice is still unclaimed. First valid server request wins. Then:

```
1 State = Carried
2 OwnerUserID = player.UserId
```

The Ice is attached/presented as being carried.

21. One-Ice Carry Limit

A player may carry:

exactly 0 or 1 Ice. Attempts to Grab another Ice while carrying one are rejected. There is no backpack storage for unthawed Ice. Successfully returned Ice is deposited into the Pen instead.

22. Carrying Ice

Carrying Ice: increases Warmth drain; does not reduce WalkSpeed; remains visually obvious. Large/high-value Ice may have stronger visual presentation but uses the same gameplay carry system.

There is no intentional DropIce gameplay request at launch. Raw Ice leaves the carry state only through Pen deposit, expedition failure/reset/death, disconnect cleanup, or explicitly documented future design.

23. Freeze / Expedition Failure

When:

```
1 CurrentWarmth <= 0
```

server executes:

```
1 FreezePlayer()
```

Result:

1. set IsFrozen = true ;
2. discard/reset carried Ice;
3. clear carry state;
4. return character to own base;
5. set CurrentWarmth = MaxWarmth;
6. remove Frozen state after reset.

Do not remove: Cash; MaxWarmth; Hearth Level; creatures; Pen Ice; Archive progress.

24. Disconnect While Carrying Ice

Carried Ice does not persist in the player's profile.

If a player disconnects while carrying normal Ice: the retrieval is considered failed; clear ownership/carry state; the normal Ice may disappear until the next Great Frost field reset.

If a player disconnects while carrying Special Ice: if the originating Special opportunity has not yet been superseded by a newer Great Frost, return the Special Ice to its authored origin SpawnPoint and make it Available again; otherwise remove it.

Do not save carried Ice into the player's profile.

25. Pen Deposit

The dedicated Thaw Machine is removed. A player carrying Ice returns to their own Pen and deposits it. Client requests:

```
1 DepositCarriedIce()
```

Server validates: player owns the carried Ice; player is inside/near own Pen; Ice state is Carried . Then the server converts the wilderness Ice into a persistent:

```
1 ThawItem
```

inside the player's profile.

26. Thaw Record

Persistent thaw record:

```
ThawID
IceType
IsSpecial

-- hidden reward
CreatureID
Rarity

-- visible jackpot lineage
WeightMultiplier

DepositedAtUnix
BaseDurationSeconds
BonusProgressSeconds
```

The deposited/thawing Ice visual uses the same derived Scale as the wilderness Ice. The record preserves the exact size lineage until reveal.

27. Thaw Progress Formula

Do not run one timer object per Ice.

Progress can be derived using real-world timestamps.

```
1 Elapsed =
2   CurrentUnixTime
3   - DepositedAtUnix
4   + BonusProgressSeconds
```

Then:

```
1 Progress =
2   clamp(Elapsed / BaseDurationSeconds, 0, 1)
```

Ready condition:

```
1 Progress >= 1
```

This supports: online progression; offline progression; Night bonuses; persistence; server restarts; without continuously updating saved countdown values.

28. Offline Thawing

Offline thawing requires no special background process. Because:

```
1 CurrentUnixTime - DepositedAtUnix
```

continues increasing while the player is offline. On join:

```
1 ThawService
```

recalculates every saved Ice. If complete:

mark/present it as READY TO CRACK . The creature is not automatically granted. The player still performs the reveal.

29. Great Frost Thaw Surge

Only players actively present when the Great Frost occurs receive the online acceleration. At Great Frost:

```
1 BonusProgressSeconds += GreatFrostThawSurge
```

for each unfinished ThawItem. Alternatively, the config may later define the bonus as a percentage of base duration. Ready items are unaffected. This gives:

offline = normal speed online during Great Frost = accelerated speed.

30. Thaw Presentation

Gameplay state remains server-side. Visual models may show:

```
1 Stage 0 - Frozen
2 Stage 1 - Frost clearing
3 Stage 2 - silhouette clearer
4 Stage 3 - heavily cracked
5 Stage 4 - Ready
```

Stages are derived from progress thresholds. Do not expose hidden rarity through these stages.

31. Unlimited Thaw Visual Layout

Gameplay thaw capacity is unlimited. Visual placement should therefore not rely on named gameplay slots. Instead, define:

```
1 Pen.ThawDisplayOrigin  
2 Pen.ThawDisplayArea
```

and generate deterministic display positions. For example:

```
1 index -> row/column placement
```

If extreme quantities create performance problems, presentation may use: model pooling; distance culling; simplified LOD; visual grouping. This must not change ownership or thaw gameplay.

32. Ready-to-Crack Interaction

When progress reaches 100%:

```
1 State = Ready
```

A prompt appears. Client requests:

```
1 BeginReveal(ThawID)
```

Server validates: player owns ThawID; item is complete; player is near own Pen; player has no conflicting reveal session. Server creates:

```
1 RevealSession
```

33. Reveal Session

Runtime structure:

```
1 SessionID  
2 PlayerUserID  
3 ThawID  
4  
5 CurrentHit = 0  
6 RequiredHits = 3  
7  
8 LastHitTime
```

It is not persisted. Disconnecting mid-reveal leaves the Ice in:

Ready state. The player can restart later.

34. Crack -> Crack -> Smash

Client requests:

```
1 Crack(SessionID)
```

Server validates: session exists; session belongs to player; expected sequence; reasonable rate limit; player remains near the Ice. Then:

```
1 CurrentHit += 1
```

Hits:

```
1 1 = Crack  
2 2 = Crack  
3 3 = Smash
```

No skill grade exists. No timing score exists.

35. Rarity Foreshadow

Hit #2 may provide server-approved presentation information. For example:

```
1 ForeshadowEffectProfile
```

The server may choose: none for Common; subtle for Rare; strong for Legendary.

Do not send:

```
1 Rarity = Legendary
```

until the final reveal.

36. Final Reveal Transaction

On Hit #3:

1. lock the ThawItem as resolving;
2. obtain stored CreatureID , Rarity , and WeightMultiplier ;

3. create exactly one new CreatureGUID copy;
4. store {CreatureID, WeightMultiplier} in CreatureCopies[CreatureGUID] ;
5. mark DiscoveredCreatures[CreatureID] = true ;
6. route that CreatureGUID into Pen or Inventory;
7. remove the ThawItem;
8. clear reveal session;
9. send reveal event including sanitized copy identity and derived visible stats.

The creature visual must inherit the exact same Scale derived from the Ice's WeightMultiplier. There is no second size roll at reveal.

This operation must be idempotent. Repeated/duplicate Crack requests must never grant multiple creature copies.

37. New Creature Routing

After reveal:

```
if Pen has empty active creature position:
    PenSlots[firstEmpty] = CreatureGUID
else:
    CreatureGUID remains in derived Inventory
```

No modal selection is required.

38. Creature Ownership

Launch ownership is per-copy:

```
CreatureCopies[CreatureGUID] = {
    CreatureID,
    WeightMultiplier,
}
```

Inventory is derived:

```
all owned CreatureGUIDs
- CreatureGUIDs currently in PenSlots
- EquippedCreatureGUID
```

Each copy may differ in Scale, Income, and Speed even when multiple copies share the same CreatureID.

Only the approved size/weight roll creates random per-copy stat variation. Do not add independent randomized Damage, Defense, skills, levels, affixes, or arbitrary stat rolls.

39. Pen Income

Pen income is calculated from the four authoritative CreatureGUID slots using one aggregate server tick. Current cadence:

```
1 second
```

For each occupied slot:

```
copy = CreatureCopies[CreatureGUID]
base = CreatureConfig[copy.CreatureID].BaseIncomePerSecond
scale = cbrt(copy.WeightMultiplier)
FinalIncome = round(base * scale^2)
```

Then:

```
TotalIncome = sum(FinalIncome for occupied PenSlots)
Cash += TotalIncome
```

No independent timer per creature. No offline Pen income. Equipped and Inventory copies earn zero. A gigantic copy may therefore be an extreme economic jackpot.

40. Selling

Sell requests operate on a specific CreatureGUID. Server validates:

- player owns the GUID;
- source state is legal;
- player is at own base;
- the copy can legally be removed.

Sell from Inventory:

```
remove CreatureCopies[CreatureGUID]
Cash += configured SellValue
```

Sell Value is not automatically size-scaled in this revision. Size-derived value currently affects Pen Income and Equipped Speed only.

If selling from Pen or Equipped is supported, clear that exact GUID allocation before removal. Frozen Archive discovery remains species-based and never reverses.

41. Frozen Archive

Persistent structure:

```
1 DiscoveredCreatures[CreatureID] = true
```

It never becomes false. The Archive UI reads from this set. Selling the last owned copy does not remove discovery.

42. Great Frost Architecture

WorldCycleService controls:

```
1 Day
2 Great Frost
```

Launch uses one shared wall-clock rhythm rather than a unique timer per server. Canonical calculation:

```
1 CycleEpochUnix = 0
2 CycleIndex = floor((CurrentUnixTime - CycleEpochUnix) / CycleLength)
3 CyclePosition = (CurrentUnixTime - CycleEpochUnix) % CycleLength
4 Day if CyclePosition < (CycleLength - GreatFrostDuration)
5 GreatFrost otherwise
```

With the initial 300-second cycle and 15-second Frost, Day occupies seconds 0-284 and Great Frost occupies seconds 285-299 of each cycle.

Server boot behavior: if the server boots during Day, populate the normal Ice field immediately for the current CycleIndex; if the server boots during Great Frost, do not populate a normal field until the next Day transition; do not retroactively award a Thaw Surge merely because the server started mid-Frost; do not retroactively spawn a Special Ice merely because the current CycleIndex qualifies as a Special cycle.

Each running server tracks LastProcessedFrostCycle so a Frost transition can never apply twice.

This keeps servers synchronized and prevents server boot/server hopping from manufacturing extra Frost bonuses or Special Ice.

43. World Cycle State

The service exposes:

```
1 CycleIndex
2 CurrentPhase
3 NextTransitionUnix
4 TimeRemaining
5 LastProcessedFrostCycle -- server runtime only
```

Clients receive timestamps rather than being trusted to maintain gameplay timing themselves. Client countdowns may interpolate locally between server corrections.

44. Great Frost Start

At the actual Day -> Great Frost transition observed by a running server: Gameplay set `GrabLockedDuringGreatFrost = true`; block new wilderness `GrabIce` confirmations during the 15-second transition; remove/reset available unclaimed shared wilderness Ice; preserve carried Ice; preserve deposited/Ready Ice; apply one Thaw Surge to eligible online players; determine whether this observed Frost transition is a Special Ice cycle. Presentation Server sends:

```
1 GreatFrostStarted
```

Clients handle: sky transition; aurora; sound; wind; frost wave; camera-safe effects. Gameplay must not depend on effects successfully rendering.

45. Ice Field Reset

Great Frost replaces short independent Ice respawn timers as the primary shared-world refresh. During reset:

```
1 Available shared wilderness Ice -> removed
```

Then, during the Frost sequence:

```
1 IceService.populateNewField()
```

selects new available `SpawnPoints` according to configuration.

At the return to Day: set `GrabLockedDuringGreatFrost = false`.

Carried Ice is unaffected. Deposited Pen Ice is unaffected. Private onboarding Ice is not part of the shared Ice field and is unaffected by Great Frost.

46. Spawn Population

Each Ice type defines:

```
1 CandidateSpawnPoints
```

```
2 ActiveSpawnCount
```

At each Great Frost:

1. gather valid candidate SpawnPoints;
2. randomly select configured active positions;
3. roll hidden Ice results;
4. create world Ice. This makes each cycle feel different while keeping placements handcrafted.

47. Special Ice

Every configurable:

```
1 X Great Frost cycles
```

an actual Frost transition observed by the running server may create:

one Special Ice opportunity unless later balance config specifies otherwise.

Special Ice is a modifier, not necessarily a sixth standard Ice Type. Example record:

```
1 IceType = Black
2 IsSpecial = true
3 RollProfile = Special
4 OriginCycleIndex = CycleIndex
5 OriginSpawnPointID = SpawnPointID
```

A Special Ice is shared and contestable. First valid server-side Grab wins.

Lifecycle: unclaimed Special Ice remains available until claimed or until the next Great Frost removes the shared field; successful Pen deposit consumes the world opportunity normally; if its carrier freezes, resets, dies, or disconnects before deposit, return it to OriginSpawnPointID only if OriginCycleIndex is still the current opportunity cycle; if a newer Great Frost has superseded that opportunity, remove it instead; carried Special Ice still obeys the global rule that carried Ice survives a Great Frost transition.

48. Special Ice Roll

Special Ice uses a dedicated rarity profile. It must guarantee:

a high-value creature opportunity with elevated rarity. Exact implementation belongs in Constants. For example, configuration may define:

```
1 SpecialRarityWeights
2 EligibleIceTypes
```

Do not hardcode Legendary.

49. Special Ice Presentation

Special Ice may be publicly recognizable through: size; runes; glow; beam; unique sound; Great Frost announcement. Its exact creature remains hidden. The server may announce:

A Strange Ice Has Awakened It should not announce:

An Ice Dragon spawned.

50. Multiplayer Ice Ownership

The wilderness is shared. When multiple players attempt to take one Ice:

first valid server-side Grab wins. Ice ownership must be explicit. No client may locally claim an Ice before server confirmation.

51. Player Bases

BaseService assigns each player:

```
1 AssignedBase
```

Launch topology uses up to four personal base plots grouped side-by-side along one shared home-camp edge. All plots face the same primary wilderness direction so players read home behind them and adventure ahead of them.

A base contains:

```
1 SpawnPoint
2 WarmZone
3 Hearth
4 PenArea
5 PenCreatureAnchors
6 ThawDisplayArea
7 OnboardingIceSpawnPoint
```

Base progression is private. Only the owner may: refill Current Warmth in that WarmZone; train Maximum Warmth there; upgrade its Hearth; manage its Pen; deposit Ice; reveal Ice; Equip/Unequip; Sell.

Another player's Hearth/WarmZone does not refill Current Warmth and does not train Maximum Warmth. It is presentation/social space only for visitors.

51A. First-Time Onboarding Ice Shared wilderness depletion must never prevent a new player from learning the core loop.

Eligibility uses existing state; no extra persistent onboarding flag is required. A player is eligible while all are true: DiscoveredCreatures is empty; ThawingIce is empty; the player carries no Ice.

When eligible and no private onboarding Ice currently exists, BaseService/IceService may create one private Regular Ice at that player's OnboardingIceSpawnPoint.

Rules: visible/interactable only for that player; uses the normal Regular rarity weights and creature pool; reward is rolled server-side and remains genuinely random; not contestable by other players; not removed by Great Frost; if the player fails before deposit, it may respawn/reappear while eligibility remains true; after successful deposit, ThawingIce is non-empty so another onboarding Ice is not created; after reveal, DiscoveredCreatures is non-empty, permanently ending this onboarding exception.

This scripts the learning opportunity, not the excitement.

52. Pen Creature Presentation

Pen movement remains cosmetic. Gameplay uses authoritative PenSlots CreatureGUID allocations, not visual position.

Each visible Pen creature should:

- use the copy's derived Scale;
- expose SPEED <value> and PEN \$<value>/s in a minimal world-space label;
- remain non-authoritative for economy or ownership.

Oversized copies are allowed to visually spill outside their nominal anchor and may overlap neighboring Pen presentation. This is intentional trophy/social presentation, not extra gameplay capacity.

Do not use heavy pathfinding or server physics to make giant pets roam.

53. Equipped Follower Presentation

Follower movement is cosmetic. Desired client behavior remains:

```
desired position behind player  
-> smooth interpolation  
-> snap/teleport if too far
```

The visible follower uses the exact equipped copy's derived Scale. At extreme scales, presentation may need camera-aware offsets so the creature remains readable without becoming a physics object.

Do not use follower CFrame to calculate Speed, Warmth, or ownership. If the Survival Pressure Experiment remains enabled, server logical creature targeting must derive bounds from the equipped copy's authoritative scale rather than trusting client follower geometry.

54. Remote Requests

Recommended shared GameplayRequest actions:

```
1 ClientReady
2
3 GrabIce
4 DepositCarriedIce
5
6 BeginReveal
7 Crack
8
9 PlaceInPen
10 RemoveFromPen
11
12 EquipCreature
13 UnequipCreature
14
15 SellCreature
16 UpgradeHearth
```

Warmth training, income, Night timing, thaw completion and Ice spawning require no client request.

55. Server-to-Client Events

GameplayEvent may carry:

```
1 StateSnapshot
2 StateUpdated
3
4 CashUpdated
5
6 WarmthUpdated
7 WarmthTrainingUpdated
8
9 CarryStateUpdated
10 Frozen
11
12 IceDeposited
13 ThawUpdated
14 ThawReady
15
16 WorldCycleUpdated
17 GreatFrostStarted
18 IceFieldReset
19 SpecialIceSpawned
20
21 RevealStarted
22 CrackConfirmed
23 RevealCreature
```

24	
25	PenUpdated
26	EquippedUpdated
27	HearthUpdated
28	
29	ShowMessage

These may share one RemoteEvent with action identifiers. Do not create a RemoteEvent for every button.

56. Remote Validation

Every gameplay request must validate: argument types; loaded player state; character existence; ownership; proximity where applicable; correct state; action order; rate limit; target existence. Never trust the client for: Cash; Warmth; Movement Speed; rarity; creature result; thaw completion; Night state; income; position; ownership.

57. Persistence Service

DataService is the only module that directly communicates with Roblox persistence.

Responsibilities: load; defaults;

schema validation; version migration; retry; autosave; disconnect save; shutdown save; session safety. If data loading fails:

do not create a blank writable profile and overwrite legitimate data. Gameplay depending on persistent state should wait until the profile has loaded successfully.

58. Join Flow

On player join:

1. load profile;
2. validate/migrate data;
3. assign base;
4. create player runtime state;
5. set CurrentWarmth = MaxWarmth ;
6. calculate Equipped Speed;
7. restore Pen creatures;
8. recalculate all ThawItem progress using current Unix time;
9. present ready/thawing Ice;
10. send state snapshot;
11. spawn player safely at base.
12. after state restore, evaluate first-time onboarding-Ice eligibility.

On any normal character respawn, set `CurrentWarmth = MaxWarmth` and return to the assigned base `SpawnPoint`. If the player had been carrying Ice, resolve the carry failure first.

59. Leave Flow

On disconnect:

1. resolve carried Ice using normal-vs-Special disconnect cleanup;
2. clear reveal runtime;
3. update persistent state;
4. save profile;
5. release base assignment;
6. remove presentation objects.

Thaw records require no logout timestamp modification because their original `DepositedAtUnix` already allows offline progression.

60. Hero Moment Technical Requirements

The five Hero Moments are not optional polish concepts. The architecture must make them easy to present.

Hero Moment 1 - First Great Frost `WorldCycleService` supplies gameplay event. `WorldCycleController` + `EffectsController` supply: sky; aurora; wind; world sound; frost wave; spawn presentation. Effects should degrade gracefully on low-end/mobile devices.

Hero Moment 2 - Impossible-Looking Ice Primarily a Level Design requirement. Technical support: handcrafted spawn points; long-distance visibility; distinct Ice presentation; silhouette profiles. No special progression system is required.

Hero Moment 3 - Barely Made It Home Server owns Warmth.

Client receives enough Warmth state to present thresholds such as:

- 1 Low
- 2 Critical
- 3 Near-Frozen

Client may add: screen frost; wind intensity; audio; warning pulses. These effects never decide when the player freezes.

Hero Moment 4 - Legendary Smash Server sends reveal result once. Client selects presentation from:

Legendary presentation may include: stronger camera response; larger VFX; unique audio; creature animation; rarity reveal. Gameplay grant occurs before/independently of cosmetic completion.

Hero Moment 5 - Endgame Speed Server provides the actual Speed advantage. Client adds: appropriate run animation; FOV response; trails; follower presentation. A Legendary must feel faster through gameplay, not only effects.

61. Creature Asset Contract

Each creature Model must use:

```
CreatureModel
+-- Root
+-- Visual Parts
```

Requirements:

- `PrimaryPart = Root`;
- standardized facing direction;
- scale-safe hierarchy compatible with `Model:ScaleTo()` or equivalent uniform model scaling;
- no gameplay scripts inside creature Models;
- gameplay code references `CreatureID`, `Model`, `Root`, and copy data rather than arbitrary visual child names.

The authored model represents that species' **1.0x base visual size**. Runtime copy Scale multiplies that authored baseline and may reach 5x.

Collision/physics used for presentation should not let an enormous cosmetic creature block the player/world unexpectedly.

62. Ice Asset Contract

Each Ice visual should support presentation states without requiring procedural destruction. Preferred:

```
1 Stage0
2 Stage1
3 Stage2
4 Stage3
5 Ready
```

or equivalent visibility swaps. Avoid complex runtime fracture physics. The final Smash can use: prebuilt debris; particles; short-lived pieces; without requiring destructible simulation.

63. Great Frost Art Direction Is Presentation, Not Gameplay

Norse-inspired fantasy elements such as: runes; aurora; magical frost; Hearth visuals; ancient ruins; must remain primarily art/presentation. Do not introduce technical dependencies such as: rune puzzles; Viking combat; magical skill trees; mythology quests; unless separately approved.

64. Prototype Technical Version

Week-1 prototype must use the same conceptual architecture so it is not throwaway code. Prototype differences:

```
1  MaxPlayers = 1
2
3  Creatures = 8
4  Rarities = 3
5
6  IceTypes =
7      Regular
8      Thick
9
10 Persistence = OFF
11
12 OfflineThawing = NOT TESTABLE
13
14 World =
15     Greybox
16
17 CreatureModels =
18     primitive Parts
19
20 HearthLevels =
21     small subset
22
23 PenCreatureSlots =
24     4
25
26 GreatFrost =
27     ON
28
29 GreatFrostThawSurge =
```

```
30     ON
31
32 SpecialIce =
33     optional / not required for core validation
```

65. Prototype Runtime Thawing

Prototype uses the same:

```
1 DepositedAt
2 BaseDuration
3 BonusProgress
```

logic in memory. Do not build a completely different countdown system merely because persistence is disabled. When launch persistence is added, the timestamp system should already be compatible.

66. Prototype Validation Priorities

Technical prototype is successful when the following work reliably:

```
1  spawn
2  -> Warmth
3  -> grab Ice
4  -> carry
5  -> freeze/fail
6  -> return Ice
7  -> Pen deposit
8  -> thaw
9  -> Night surge
10 -> ready
11 -> Crack Crack Smash
12 -> grant creature
13 -> Pen
14 -> income
15 -> Equip
16 -> clearly faster movement
17 -> Heater upgrade
18 -> faster Warmth training
```

Before expanding content, this complete loop should survive repeated testing.

67. Critical State Invariants

The server must always preserve:

```
1 0 <= CurrentWarmth <= MaxWarmth
```

```
1 MaxWarmth >= starting value
```

```
1 0 or 1 carried Ice per player
```

```
1 0 or 1 owner per wilderness Ice
```

```
1 0 or 1 Equipped creature
```

```
1 max 4 active Pen creatures
```

```
1 PenCopies + EquippedCopies  
2 <= OwnedCreatureCount
```

There is no gameplay maximum for:

```
1 ThawingIce count
```

And only the server may mutate:

```
1 Cash  
2 MaxWarmth  
3 HearthLevel  
4 OwnedCreatures  
5 PenSlots  
6 EquippedCreature  
7 DiscoveredCreatures  
8 ThawingIce  
9 Ice ownership  
10 Ice reward result
```

68. Explicit Technical Non-Goals

Do not implement for launch without a separate design decision:

- pickaxe/mining systems;
- creature-specific traversal abilities;
- bosses;
- large weapon/combat progression trees;
- separate Speed training tree;
- stamina/hunger;
- additional currencies;
- prestige/rebirth;
- Daily Quests/login calendars;
- randomized per-copy combat stats beyond the approved inherited size roll;
- offline Pen income;
- purchasable Thaw Slots;

- separate Thaw Machine;
- tap-to-accelerate Thawing;
- procedural Ice coordinate spawning;
- physics-heavy pet AI;
- complex destructible Ice simulation.

Canonical per-copy exception: every creature copy may vary through its inherited Ice `WeightMultiplier`, which determines visual Scale and modifies Pen Income and Equipped Speed. This variation is intentional and must persist.

69. Implementation Priority

If technical scope becomes constrained, protect in this order:

1. Server-authoritative expedition loop

Warmth -> Grab -> Carry -> Freeze/Return. 2. Pen thaw lifecycle

Deposit -> time -> offline progression -> Ready. 3. Reliable reveal transaction

Crack -> Crack -> Smash -> creature grant. 4. Progression

Pen income -> Hearth -> Warmth training -> Equipped Speed. 5. Great Frost

world reset + Thaw Surge. 6. Persistence

especially MaxWarmth, creatures and ThawItems. 7. Multiplayer safety

Ice ownership, bases, server validation. 8. Hero Moment presentation

Presentation can be simplified, but the five Hero Moments should remain recognizable.

70. AI Implementation Rule

When an AI coding assistant implements this project:

1. read the current task;
2. consult this Technical Specification;
3. consult Implementation Constants for exact values;
4. consult the GDD only when player-facing intent needs clarification;
5. implement only the requested task and required dependencies;
6. test;
7. fix;
8. confirm success before moving to the next task.

Do not independently add: systems; currencies; progression; libraries; remotes; abstraction layers; because they "might be useful later." Prefer:

smallest robust implementation that satisfies the canonical design.

71. Canonical Technical Summary

The entire game should technically reduce to a small number of authoritative systems:

WarmthService trains and drains endurance.

CreatureService handles ownership, Pen income, Equipped Speed, selling and discovery.

IceService creates the wilderness opportunities and controls retrieval.

ThawService turns successfully recovered Ice into persistent real-time anticipation.

WorldCycleService transforms the wilderness periodically, accelerates online thawing and creates Special Ice opportunities.

DataService ensures that the player's creatures, Warmth, Hearth and unfinished Ice remain meaningful when they return. Everything visual-the Great Frost, giant Ice, low-Warmth tension, Legendary Smash and high-Speed fantasy-should amplify those systems rather than create new ones.

The technical implementation succeeds when the game can deliver "I saw it -> I risked it -> I got it home -> I waited for it -> I smashed it open -> now I'm stronger" reliably, securely, and repeatedly.